

Лекция № 3

Ю.Т. Каганов, к.т.н., доцент, кафедра ИУ-5

Методы оптимизации в машинном обучении

План

1. Обучение. Понятие оптимальности.
2. Математическое программирование. Основные понятия.
3. Методы оптимизации в машинном обучении.
4. Градиентные методы решения задач в МО.
5. Задачи выпуклого программирования и теорема Куна-Таккера.
6. Двойственные задачи.
7. Методы штрафных функций.
8. Пример: использование теоремы Куна-Таккера в методе опорных векторов.

Обучение

Обучение — это процесс приобретения новых знаний, навыков, умений или поведения в результате изучения, опыта или практики.

Обучение как философская категория

В философии обучение рассматривается не просто как процесс усвоения знаний, а как фундаментальный феномен человеческого существования, связанный с познанием, становлением личности и взаимодействием с миром.

Гносеологический аспект (теория познания)

1. **Рационализм** (Декарт, Кант) – обучение как процесс логического осмысления и открытия истин разумом.
2. **Эмпиризм** (Локк, Юм) – знание возникает из опыта, обучение основано на чувственном восприятии.
3. **Конструктивизм** (Пиаже, Выготский) – знание не "передаётся", а конструируется в процессе взаимодействия с миром.

Машинное обучение (Machine Learning, ML)

Машинное обучение (Machine Learning, ML) — это подраздел искусственного интеллекта (ИИ), изучающий методы, которые позволяют компьютерам *автоматически обучаться* на данных *без явного программирования*. В отличие от традиционного ПО, где правила жёстко заданы, ML-алгоритмы выявляют закономерности и *самостоятельно улучшают* свои прогнозы или решения.

◆ Обучение на данных

ML основано на **статистических закономерностях**: алгоритм анализирует входные данные (например, изображения, текст, числа) и находит скрытые взаимосвязи.

Чем больше и качественнее данные, тем лучше модель "учится".

◆ Адаптивность

Модель не просто запоминает примеры, а **обобщает** закономерности для работы с новыми, неизвестными данными.

Пример: распознавание рукописных цифр (даже если почерк новый).

Сущность машинного обучения — в **автоматическом выявлении закономерностей из данных** и способности к *самооптимизации*. Оно меняет парадигму программирования, заменяя "жёсткую логику" на адаптивные модели, но поднимает вопросы интерпретируемости и этики.

Понятие оптимальности

Понятие оптимальности предполагает наличие критериев оценки качества решения. При этом должно достигаться экстремальное значение этих критериев при удовлетворении некоторых ограничивающих условий.

С точки зрения математики задачи, поставленные таким образом, называются экстремальными. Это задачи, для которых ищется экстремум – т.е. минимум или максимум некоторой функции или функционала при заданных ограничениях. Решение задачи состоит в поиске экстремума путем модификации параметров таким образом, чтобы целевая функция приближалась к экстремуму.

$$\begin{aligned} &extr (min, max) f(x, y, \theta) \\ &\theta_{i+1} = \theta_i + \Delta\theta_i \\ &f(x, y, \theta_{i+1}) > f(x, y, \theta_i) \end{aligned}$$

Математическое программирование:

ОСНОВНЫЕ ПОНЯТИЯ

Математическое программирование — это раздел математики, занимающийся поиском экстремумов (максимумов или минимумов) функций при заданных ограничениях. Оно широко применяется в экономике, технике, управлении и других областях для оптимизации решений.

Основные понятия

1.1. Оптимизационная задача

Любая задача математического программирования включает:

- **Целевую функцию** $f(x)$ — функцию, которую нужно максимизировать или минимизировать.
- **Переменные решения** $x = (x_1, x_2, \dots, x_n)$ — параметры, которые можно изменять.
- **Ограничения** — условия, которым должны удовлетворять переменные (равенства или неравенства).

$$\max \text{ (или } \min) f(x),$$

$$\text{Общий вид задачи: } g_i(x) \leq 0, i=1, \dots, m,$$

$$h_j(x) = 0, j=1, \dots, k.$$

Математическое программирование:

ОСНОВНЫЕ ПОНЯТИЯ

В зависимости от вида целевой функции и ограничений выделяют:

1. Лине́йное программирование (ЛП)

1. Целевая функция и ограничения линейны.
2. Пример: задача оптимального распределения ресурсов.

2. Не́линейное программирование (НЛП)

1. Целевая функция или ограничения нелинейны.
2. Пример: задача минимизации квадратичной функции.

3. Це́лочисленное программирование

1. Переменные принимают только целые значения.
2. Пример: задача о рюкзаке или коммивояжёре.

4. Динамическое программирование

1. Оптимизация многошаговых процессов (метод Беллмана).

5. Вы́пуклое программирование

1. Целевая функция выпукла, ограничения задают выпуклое множество.

6. Стохастическое программирование

1. Учитывает случайные параметры в модели.

7. Неопределенное программирование

1. Использование теории нечетких множеств.

Математическое программирование:

ОСНОВНЫЕ ПОНЯТИЯ

Допустимое и оптимальное решение

- **Допустимое решение** — удовлетворяет всем ограничениям.
- **Оптимальное решение** — допустимое решение, дающее экстремум целевой функции.

Градиент и условия оптимальности

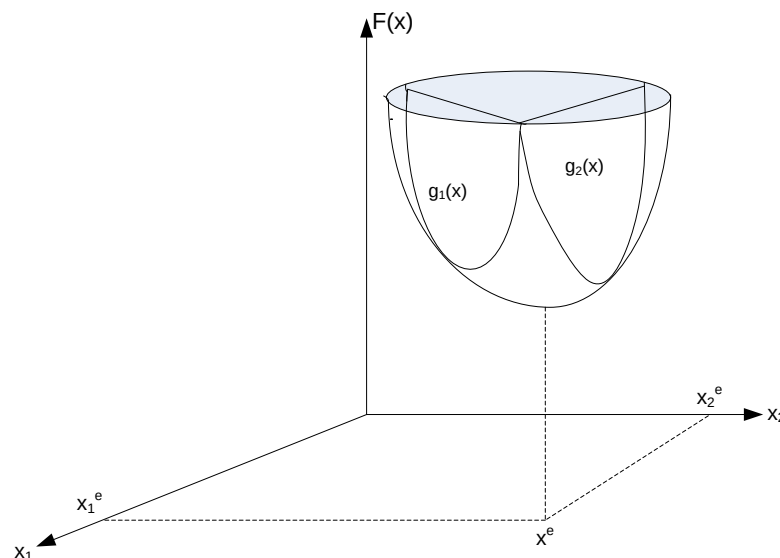
- **Градиент $\nabla f(x)$** — вектор частных производных, указывающий направление наискорейшего роста функции.
- **Условия Куна-Таккера** — обобщение метода множителей Лагранжа для задач с ограничениями.

Методы решения

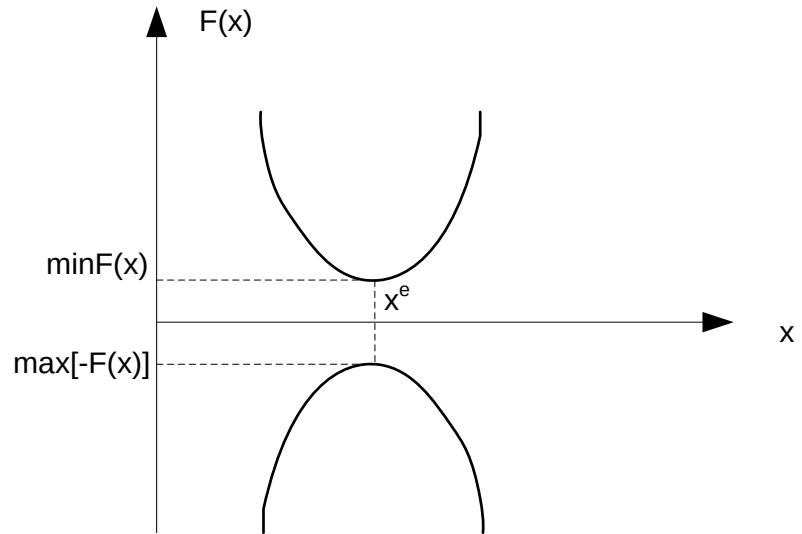
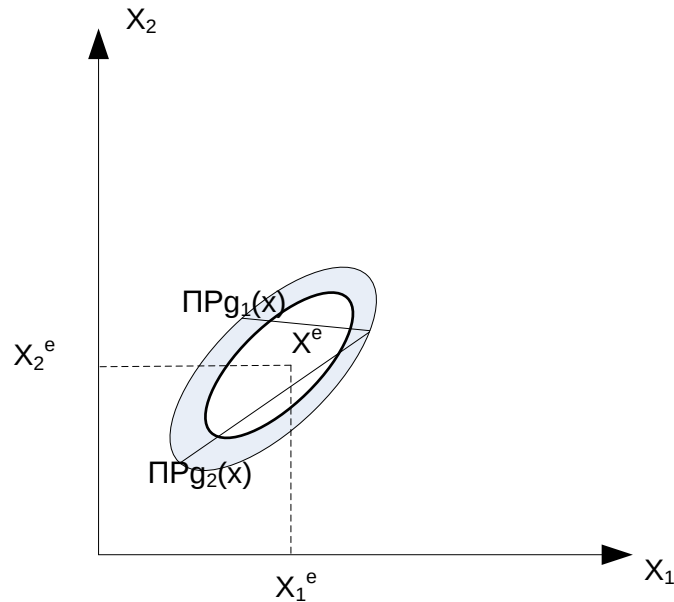
- **Симплекс-метод** (для линейных задач).
- **Метод Ньютона, градиентный спуск** (для нелинейных задач).
- **Метод ветвей и границ** (для целочисленных задач).
- **Динамическое программирование** (для многоэтапных задач).

Понятие оптимальности

- Решение задачи состоит в поиске экстремума целевой функции (критерия оптимальности) при заданных ограничениях и уравнениях состояния.



Понятие оптимальности



Задача минимизации функции (функционала) всегда может быть переформулирована как задача максимизации и наоборот.

Методы оптимизации в машинном обучении

Методы оптимизации в машинном обучении (МО) играют ключевую роль, так как большинство алгоритмов сводятся к задаче минимизации функции потерь (loss function). Их специфика обусловлена особенностями данных, моделей и вычислительных процессов в ML.

Методы оптимизации в машинном обучении

- **Основные лосс-функции в машинном обучении.**

Для задач регрессии

1. Mean Squared Error (MSE, Среднеквадратичная ошибка)
2. Mean Absolute Error (MAE, Средняя абсолютная ошибка)

Для задач классификации

1. Binary Cross-Entropy (Бинарная кросс-энтропия)
2. Categorical Cross-Entropy (Категориальная кросс-энтропия)

Для задач ранжирования и других

1. KL-Divergence (Kullback-Leibler Divergence)
2. Contrastive Loss

Лосс-функции

Задачи регрессии

Mean Squared Error (MSE) - Среднеквадратичная ошибка

$$L(y, \hat{y}) = 1/n \sum (y_i - \hat{y}_i)^2$$

Mean Absolute Error (MAE) - Средняя абсолютная ошибка

$$L(y, \hat{y}) = 1/n \sum |y_i - \hat{y}_i|$$

Huber Loss - Потери Хубера

$$L(y, \hat{y}) = \left\{ \begin{array}{l} \frac{1}{2}(y - \hat{y})^2, \text{ если } |y - \hat{y}| \leq \delta \\ \delta|y - \hat{y}| - \frac{1}{2}\delta^2, \text{ иначе} \end{array} \right\}$$

Root Mean Squared Error (RMSE)

$$L(y, \hat{y}) = \sqrt{1/n \sum (y_i - \hat{y}_i)^2}$$

Mean Squared Logarithmic Error (MSLE)

$$L(y, \hat{y}) = 1/n \sum (\log(y_i + 1) - \log(\hat{y}_i + 1))^2$$

Лосс-функции

Задачи классификации

Binary Cross-Entropy (Log Loss) - Бинарная перекрёстная энтропия

$$L(y, \hat{y}) = -1/n \sum [y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)]$$

Categorical Cross-Entropy - Категориальная перекрёстная энтропия

$$L(y, \hat{y}) = -\sum \sum y_{ij} \cdot \log(\hat{y}_{ij}) \quad i \quad j$$

где j - индекс класса

Sparse Categorical Cross-Entropy

$$L(y, \hat{y}) = -\sum \log(\hat{y}_{i, y_i}) \quad i$$

где y_i - индекс истинного класса

Hinge Loss (для SVM)

$$L(y, \hat{y}) = \sum \max(0, 1 - y_i \cdot \hat{y}_i) \quad i$$

где $y_i \in \{-1, +1\}$

Методы оптимизации в машинном обучении

- **Основные методы оптимизации**

Градиентные методы первого порядка

1. **SGD (Stochastic Gradient Descent)** – базовый метод, обновляет параметры по градиенту на мини-батче:

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t, x_i)$$

Проблемы: чувствительность к learning rate (η), медленная сходимость.

2. **SGD with Momentum** – добавляет инерцию для ускорения сходимости:

$$v_{t+1} = \gamma v_t + \eta \nabla L(\theta_t)$$

$$\theta_{t+1} = \theta_t - v_{t+1}$$

(γ – коэффициент затухания, обычно ~ 0.9).

3. **Nesterov Accelerated Gradient (NAG)** – улучшенный momentum, учитывающий будущий градиент.

NAG (Nesterov Accelerated Method)

Основные формулы NAG:

$$v_t = \mu \cdot v_{\{t-1\}} - \eta \cdot \nabla f(\theta_{\{t-1\}} + \mu \cdot v_{\{t-1\}})$$

$$\theta_t = \theta_{\{t-1\}} + v_t$$

Где:

θ_t — параметры модели на шаге t

v_t — вектор скорости (импульс) на шаге t

μ — коэффициент импульса (обычно 0.9)

η — скорость обучения (*learning rate*)

$\nabla f(\cdot)$ — градиент функции потерь

$$\tilde{\theta} = \theta_{\{t-1\}} + \mu \cdot v_{\{t-1\}} \text{ Предсказанная позиция}$$

$$v_t = \mu \cdot v_{\{t-1\}} - \eta \cdot \nabla f(\tilde{\theta}) \text{ Обновление скорости}$$

$$\theta_t = \theta_{\{t-1\}} + v_t \text{ Обновление параметров}$$

Алгоритм NAG

Вход:

- Начальные параметры θ_0
- Скорость обучения η
- Коэффициент импульса μ (обычно 0.9)
- Функция потерь $f(\theta)$

Инициализация:

- $v_0 = 0$ (начальная скорость)
- $t = 0$

Итерации:

1. ПОКА не достигнута сходимость:
2. Вычислить предсказанную позицию: $\tilde{\theta} = \theta_t + \mu \cdot v_t$
3. Вычислить градиент в предсказанной позиции: $g_t = \nabla f(\tilde{\theta})$
4. Обновить скорость: $v_{\{t+1\}} = \mu \cdot v_t - \eta \cdot g_t$
5. Обновить параметры: $\theta_{\{t+1\}} = \theta_t + v_{\{t+1\}}$
6. $t = t + 1$
7. ВЕРНУТЬ θ_t

Ключевые отличия NAG

Преимущество: NAG "заглядывает вперёд", вычисляя градиент в точке, куда мы движемся, а не там, где находимся. Это даёт:

- Лучшую коррекцию траектории
- Более быстрое торможение перед минимумом
- Уменьшение осцилляций

Методы оптимизации в машинном обучении

- **Адаптивные методы (Adaptive Optimizers)**

Учитывают историю градиентов для адаптивного изменения learning rate:

AdaGrad – адаптирует шаг для каждого параметра:

$$\theta_{t+1} = \theta_t - \eta G_t + \epsilon \nabla L(\theta_t)$$

(G_t – сумма квадратов прошлых градиентов).

Проблема: learning rate может слишком уменьшиться.

RMSProp – исправляет AdaGrad, используя экспоненциальное скользящее среднее:

$$G_t = \beta G_{t-1} + (1 - \beta)(\nabla L(\theta_t))^2$$

- **Adam (Adaptive Moment Estimation)** – комбинирует momentum и RMSProp:

" $m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla L(\theta_t)$ " (скользящее среднее градиента)

$v_t = \beta_2 v_{t-1} + (1 - \beta_2)(\nabla L(\theta_t))^2$ (скользящее среднее квадратов)

$$\theta_{t+1} = \theta_t - \eta \cdot m_t v_t + \epsilon$$

2.3. Методы второго порядка

- Используют информацию о кривизне (матрица Гессе H), но сложны для больших моделей:
- **Метод Ньютона** – $\theta_{t+1} = \theta_t - H^{-1} \nabla L(\theta_t)$
- **L-BFGS** – квази-ньютоновский метод, подходит для небольших задач.

МЕТОД L-BFGS-B

(Limited-memory BFGS with Bounds)

- Квази-ньютоновский метод с ограниченной памятью
- Модификация BFGS для решения двух проблем:
- Высокое потребление памяти
- Поддержка граничных ограничений
- Limited Memory (L): Вместо хранения полной матрицы B_k , хранятся только m последних пар векторов $\{s_i, y_i\}$ (обычно $m \approx 10$).
- Произведение $B_k \nabla f$ вычисляется на лету с помощью двухуровневой рекурсии.
- Поддержка ограничений вида: $l_i \leq x_i \leq u_i$ (box constraints)

Алгоритм

Используется метод проекции градиента для определения активного множества (переменных на границах)

Вычисляется точка Коши (Cauchy point) — движение по антиградиенту с учетом границ

В подпространстве свободных переменных решается задача минимизации квадратичной модели с L-BFGS аппроксимацией

Линейный поиск с соблюдением границ

L-BFGS - Limited Memory BFGS

- **Проблема BFGS:** $O(n^2)$ память недопустимо для больших n
-
- **Идея L-BFGS:**
- Не хранить матрицу B_t явно
- Хранить только m последних пар векторов $\{s_i, y_i\}$
- Вычислять $B_t \nabla f(w_t)$ неявно
-
- **Алгоритм L-BFGS:**
- Память: $O(mn)$, обычно $m = 5-20$
- Время: $O(mn)$ на итерацию

Сравнение методов и практические рекомендации

Метод	Сложность	Скорость	Параметры	Скорость сходимости	Применение
SGD	$O(n)$	Низкие	η	Медленная	Простые задачи
Momentum	$O(n)$	Низкие	η, γ	Средняя	Общее
NAG	$O(n)$	Низкие	η, γ	Средняя+	Выпуклая оптимизация
AdaGrad	$O(n)$	Средние	η	Может застрять	Разреженные данные
RMSProp	$O(n)$	Средние	η, β	Хорошая	RNN
Adam	$O(n)$	Средние	η, β_1, β_2	Хорошая	Стандарт DL
L-BFGS	$O(mn)$	Высокие	m	Быстрая	Малые/средние сети

Практические рекомендации:

- По умолчанию:
- 🏆 Adam ($lr=0.001$) - для большинства задач
- 🥈 SGD + Momentum ($lr=0.01-0.1$, $momentum=0.9$) - для хорошей генерализации

Специальные случаи:

- CV (ResNet, etc.): SGD + Momentum + LR schedule
- NLP (Transformers): Adam/AdamW
- RNN: RMSProp или Adam
- Мало данных: L-BFGS
- **Советы:**
- Начать с Adam для быстрого прототипирования
- Для production: попробовать SGD + Momentum с тщательным подбором LR

Методы условной оптимизации

(методы математического программирования с учетом ограничений)

Методы нелинейного программирования.

1. **Прямые методы** – это методы, где последовательно решаются задачи оптимизации таким образом, что на каждом этапе процесс решения не выходит за пределы активных ограничений.
 - 1.1. Методы проекции градиента.
 - 1.2. Методы решения уравнений Куна-Таккера.
2. **Двойственные методы** – ряд методов, которые строят вспомогательную функцию по аналогии с двойственной функцией Лагранжа.
 - 2.1. **Методы штрафных функций**
 - 2.1.2. Методы внешних штрафов.
 - 2.1.3. Методы внутренних штрафов (метод барьерных функций).
 - 2.1.4. Метод последовательной безусловной оптимизации (МПБМ-SUMT).
 - 2.2. **Лагранжевы методы.**
 - 2.2.1. Метод Удзавы и Эрроу-Гурвица.
 - 2.2.2. Методы модифицированных функций Лагранжа.

Задача оптимизации с ограничениями

Постановка задачи

1. Общая задача оптимизации:

- $\min f(w)$
- при ограничениях: $g_i(w) \leq 0, i = 1, \dots, m$ (неравенства)
 $h_j(w) = 0, j = 1, \dots, p$ (равенства)

Примеры в ML:

1. SVM:

$$\min \frac{1}{2} \|w\|^2$$

w, b

$$\text{при: } y_i(w^T x_i + b) \geq 1, \forall i$$

2. Оптимизация с регуляризацией:

$$\min L(w)$$

$$W \text{ при: } \|w\|_1 \leq C \text{ (LASSO)}$$

3. Ограниченная оптимизация: $\min f(w)$ при: $w_i \geq 0, \sum w_i = 1$ (например, веса портфеля)

Функция Лагранжа

Определение:

- $\mathcal{L}(w, \alpha, \beta) = f(w) + \sum_i \alpha_i g_i(w) + \sum_j \beta_j h_j(w)$

где:

- $\alpha = (\alpha_1, \dots, \alpha_m)$ - множители Лагранжа для неравенств
- $\beta = (\beta_1, \dots, \beta_p)$ - множители Лагранжа для равенств
- $\alpha_i \geq 0$ для всех i

Свойство:

- $\max_{\alpha \geq 0, \beta} \mathcal{L}(w, \alpha, \beta) = \begin{cases} f(w), & \text{если } w \text{ допустимо} \\ +\infty, & \text{иначе} \end{cases}$

ЗАДАЧА НЕЛИНЕЙНОГО ПРОГРАММИРОВАНИЯ

$$\begin{array}{ll} f(x) \rightarrow \min; & f(x) \rightarrow \min; \\ h_j(x) = 0, \quad j = 1, 2, \dots, l; & h(x) = 0, \\ g_k(x) \leq 0, \quad k = l + 1, \dots, m. & g(x) \leq 0. \end{array} \quad \text{Или}$$

$$L(x, \lambda, \mu) = \lambda_0 f(x) + \sum_{j=1}^l \lambda_j h_j(x) + \sum_{k=l+1}^m \mu_k g_k(x)$$

Элементы X , удовлетворяющие ограничениям задачи называются **допустимыми решениями**.

Терема Куна-Таккера (Каруша-Джона)

Если точка x^* есть решение задачи математического программирования, то найдутся такие числа $\lambda_0^*, \lambda_1^*, \lambda_2^*, \dots, \lambda_l^*$, и $\mu_{l+1}^*, \dots, \mu_m^*$ не все равные нулю, что

$$\lambda_0^* \nabla f(x^*) + \sum_{j=1}^l \lambda_j^* \nabla h_j(x^*) + \sum_{k=l+1}^m \mu_k^* \nabla g_k(x^*) = 0,$$

причем $\lambda_0^* \geq 0$ а все $\mu_k^* \geq 0$,

при этом $\mu_k^* g_k(x^*) = 0, \quad k = l + 1, \dots, m$ - называется условием **дополняющей нежесткости**.

Теорема Каруша-Куна-Таккера

- **Теорема Куна-Таккера** (Kuhn-Tucker) — это фундаментальный результат в теории математической оптимизации, обобщающий метод множителей Лагранжа на задачи с ограничениями в форме неравенств. Она играет ключевую роль в машинном обучении, особенно в задачах обучения с ограничениями, SVM (Support Vector Machines) и других оптимизационных моделях.

Условия Каруша-Куна-Таккера (ККТ)

Теорема КТТ (достаточные условия для выпуклой задачи):

- Если f , g_i выпуклы, h_j аффинны, и выполнены условия регулярности, то w^* оптимально $\Leftrightarrow$

1. Стационарность:

- $\nabla_m \mathcal{L}(w, \alpha, \beta^*) = 0 \Leftrightarrow \nabla f(w) + \sum \alpha_i \nabla g_i(w) + \sum \beta_j \nabla h_j(w^*) = 0$

2. Допустимость прямой задачи:

- $g_i(w^*) \leq 0, \forall i$
- $h_j(w^*) = 0, \forall j$

3. Допустимость двойственной задачи:

- $\alpha_i^* \geq 0, \forall i$

4. Комплементарность (complementary slackness – условие дополняющей нежесткости):

- $\alpha_i g_i(w) = 0, \forall i$

Интерпретация комплементарности:

- Если ограничение не активно ($g_i(w) < 0$), то $\alpha_i = 0$
- Если $\alpha_i > 0$, то ограничение активно ($g_i(w) = 0$)

Двойственность в задачах оптимизации и в машинном обучении.

- Двойственность — это важный концепт как в теории оптимизации, так и в машинном обучении. Она позволяет получить дополнительную информацию о проблемах оптимизации и может значительно упростить вычисления.

Двойственность

Исходная задача (primal):

- $p^* = \min \max \mathcal{L}(w, \alpha, \beta)$
- $w \quad \alpha \geq 0, \beta$

Двойственная задача (dual):

- $d^* = \max \min \mathcal{L}(w, \alpha, \beta)$
- $\alpha \geq 0, \beta \quad w$

Теорема слабой двойственности:

- $d \leq p$

Двойственность в задачах ОПТИМИЗАЦИИ

- В контексте линейного программирования, каждая задача (прямая) имеет соответствующую двойственную задачу. Если у нас есть задача минимизации:
- $\min c^T x$
- при условии $Ax \leq b, x \geq 0$,
- то соответствующая двойственная задача будет выглядеть следующим образом:
- $\max b^T y$
- при условии $A^T y = c, y \geq 0$.
- Здесь:
- c — вектор коэффициентов целевой функции,
- A — матрица ограничений,
- b — вектор правых частей ограничений,
- x — вектор переменных прямой задачи,
- y — вектор двойственных переменных (множителей Лагранжа $y = \lambda$)

РЕГУЛЯРИЗАЦИЯ

Борьба с переобучением

Проблема переобучения:

$$\min L(w) = (1/n) \sum \text{Loss}(y_i, f(x_i; w))$$

- Модель "запоминает" обучающие данные
- Плохая генерализация на новых данных

Решение - регуляризация:

- $\min L(w) + \lambda R(w)$
- где $R(w)$ - регуляризатор, $\lambda > 0$ - сила регуляризации

Интерпретации:

1. Байесовская:

- $R(w)$ соответствует prior $p(w)$
- MAP оценка: $w^* = \operatorname{argmax} p(w|X, y) = \operatorname{argmax} p(y|X, w)p(w)$

2. Ограничение:

$\min L(w)$ при $R(w) \leq C$ эквивалентно (по ККТ) задаче с λ

3. Regularization by penalty:

"Штраф" за сложность модели

L2 регуляризация

Гребневая регрессия (Ridge Regression)

Определение:

- $R(w) = \frac{1}{2} \|w\|_2^2 = \frac{1}{2} \sum w_i^2$

Задача оптимизации:

- $\min (1/2n) \|Xw - y\|^2 + (\lambda/2) \|w\|^2$

Аналитическое решение:

- $w^* = (X^T X + \lambda I)^{-1} X^T y$

Градиент:

- $\nabla(L + \lambda R) = \nabla L + \lambda w$

Обновление в SGD:

- $w_{t+1} = w_t - \eta(\nabla L(w_t) + \lambda w_t) = (1 - \eta\lambda)w_t - \eta\nabla L(w_t)$

Свойства:

- ✓ Гладкая, дифференцируемая
- ✓ Аналитическое решение (для линейных моделей)
- ✓ Штрафует большие веса $\rightarrow$ предотвращает переобучение
- ✓ Улучшает обусловленность ($X^T X + \lambda I$ лучше обусловлена)
- ⚠ Не обнуляет веса (feature selection невозможен)

L1 регуляризация – LASSO

Least Absolute Shrinkage and Selection Operator

Определение:

$$R(w) = \|w\|_1 = \sum |w_i|$$

Задача оптимизации:

- $\min (1/2n) \|Xw - y\|^2 + \lambda \|w\|$

Субградиент:

- $\partial \|w\|_1 = \text{sign}(w)$ для $w \neq 0$ $\in [-1, 1]$ для $w = 0$

Проксимальный оператор (soft thresholding):

- $w_{i,t+1} = \text{sign}(w_{i,t}) \max(|w_{i,t}| - \eta\lambda, 0)$

Ключевое свойство: Sparsity (разреженность)

- L1 обнуляет некоторые веса!
- Автоматический выбор признаков
- Интерпретируемые модели

Сравнение L1 vs L2

Свойство	L2 (Ridge)	L1 (LASSO)
-----	-----	-----
Форма	Круг	Ромб
Решение	Аналитическое	Численное
Веса	Малые	Разреженные (нули)
Feature selection	Нет	Да
Устойчивость	Да	Меньше

Двойственность в машинном обучении

- В машинном обучении концепция двойственности часто используется в контексте задач, связанных с методом опорных векторов (SVM). В SVM мы стремимся максимизировать зазор между классами, что может быть сформулировано как задача оптимизации:
- $\min \frac{1}{2} \|w\|^2$
- При условии $y_i(w^T x_i + b) \geq 1, \forall i,$
- где w — вектор весов, b — смещение, x_i — входные данные, а y_i — метки классов.
- Соответствующая двойственная задача будет:
- $\max \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^m \alpha_i \alpha_j y_i y_j (x_i^T x_j)$
- при условии: $\sum_{i=1}^n \alpha_i y_i = 0, \alpha_i \geq 0.$

Метод опорных векторов (SVM)

- **1. Основные формулы и алгоритм**
- **Линейный SVM для бинарной классификации**
- **Цель:** Найти гиперплоскость, максимально разделяющую классы.
- **Уравнение разделяющей гиперплоскости:**
- $w^T x + b = 0$
- где w - вектор нормали, b - смещение
- **Классификация точки:**
- $y = \text{sign}(w^T x + b)$
- **Задача оптимизации**
- **Прямая задача (Hard Margin):**
- $\min\{w, b\} \left(\frac{1}{2}\right) ||w||^2$
-
- при условиях: $y_i(w^T x_i + b) \geq 1$, для всех i

Метод опорных векторов (SVM)

- **Soft Margin SVM** (с допустимыми ошибками):
- $\min\{w, b\} \left(\frac{1}{2} \right) \|w\|^2 + C \sum \xi_i$
-
- при условиях:
- $-y_i(w^T x_i + b) \geq 1 - \xi_i$
- $\xi_i \geq 0$
- где:
- **C** - параметр регуляризации (баланс между максимизацией зазора и минимизацией ошибок)
- ξ_i - переменные слабины (slack variables)

Метод опорных векторов (SVM)

- 2. "Ядерный трюк" (Kernel Trick)
- Концепция
- Вместо явного отображения данных в высокоразмерное пространство $\varphi(x)$, используем **функцию ядра**:
 - $K(x_i, x_j) = \varphi(x_i)^T \varphi(x_j)$
 - Двойственная задача с ядром
 - $\max_{\alpha} \sum \alpha_i - \left(\frac{1}{2}\right) \sum \sum \alpha_i \alpha_j y_i y_j K(x_i, x_j)$
 - Классификация:
 - $f(x) = \text{sign}(\sum \alpha_i y_i K(x_i, x) + b)$

Метод опорных векторов (SVM)

- Популярные функции ядра
- Линейное ядро:
 - $K(x, x') = x^T x'$
- Полиномиальное ядро:
 - $K(x, x') = (\gamma x^T x' + r)^d$
- **RBF** (Радиальная базисная функция, Гауссово ядро):
 - $K(x, x') = \exp(-\gamma \|x - x'\|^2)$
- Сигмоидное ядро:
 - $K(x, x') = \tanh(\gamma x^T x' + r)$

Метод опорных векторов (SVM)

- **Преимущества ядерного трюка**
 - . Не нужно явно вычислять $\phi(x)$
 - . Работа в бесконечномерных пространствах
 - . Вычислительная эффективность

Метод опорных векторов (SVM)

3. Алгоритм обучения *SVM*

- Шаги:

1. Выбор ядра и параметров (C , γ и др.)
2. Формирование задачи оптимизации с матрицей ядер
3. Решение задачи квадратичного программирования:
 1. *Sequential Minimal Optimization (SMO)*
 2. *Gradient descent methods*
 3. *Coordinate descent*
4. Определение опорных векторов ($\alpha_i > 0$)
5. Вычисление смещения b :

$$b = y_s - \sum \alpha_i y_i K(x_i, x_s)$$

для опорного вектора x_s

Метод опорных векторов (SVM)

- **Двойственная задача (Dual Problem)**
- $\max_{\alpha} \sum \alpha_i - \left(\frac{1}{2}\right) \sum \sum \alpha_i \alpha_j y_i y_j (x_i^T x_j)$
-
- при условиях:
- $- 0 \leq \alpha_i \leq C$
- $-\sum \alpha_i y_i = 0$
- **Решение:**
- $w = \sum \alpha_i y_i x_i$
- **Опорные векторы** - объекты с $\alpha_i > 0$

SVM vs Нейронные сети

Аспект	SVM	Нейронные сети
-----	-----	-----
Оптимизация	Выпуклая (QP)	Невыпуклая
Решение	Глобальный оптимум	Локальный минимум
Гиперпараметры	C, kernel params	Много (архитектура, η , ...)
Интерпретируемость	Опорные векторы	Черный ящик
Масштабируемость	Плохая ($O(n^2)$)	Хорошая (SGD)
Feature engineering	Важен выбор ядра	Автоматический
Малые данные	✓ Хорошо	⚠ Переобучение
Большие данные	✗ Проблемы	✓ Отлично
Высокая размерность	✓ Kernel trick	✓ Deep learning

Когда использовать SVM:

- ✓ Малые/средние датасеты (< 10K примеров)
- ✓ Нужны теоретические гарантии
- ✓ Важна интерпретируемость
- ✓ Табличные данные с хорошими признаками

Когда использовать нейросети:

- ✓ Большие данные (> 100K примеров)
- ✓ Сложные структурированные данные (изображения, текст, аудио)
- ✓ Нужна автоматическая feature extraction
- ✓ Есть вычислительные ресурсы (GPU)

Двойственность в машинном обучении

Двойственность в задачах оптимизации и машинном обучении предоставляет мощные инструменты для анализа и решения задач. Она позволяет не только находить оптимальные решения, но и понимать взаимосвязи между различными задачами.

МЕТОДЫ ОПТИМИЗАЦИИ В МАШИННОМ ОБУЧЕНИИ

1. Goodfellow I., Bengio Y., Courville A. "Deep Learning"

MIT Press, 2016. **Библия глубокого обучения.** Глава 8: Optimization for Training Deep Models. Охватывает SGD, Adam, momentum, batch normalization. Доступна бесплатно: www.deeplearningbook.org.

2. Bottou L., Curtis F.E., Nocedal J. "Optimization Methods for Large-Scale Machine Learning"

SIAM Review, 2018, Vol. 60, No. 2. **Обзорная статья (100+ страниц).** Stochastic gradient methods, mini-batch methods. От классики до современных подходов Обязательно к прочтению!

3. Ruder S. "An Overview of Gradient Descent Optimization Algorithms"

arXiv:1609.04747, 2016. Популярный обзор алгоритмов градиентного спуска. SGD, Momentum, Nesterov, Adagrad, RMSprop, Adam. Практичный и понятный Доступен бесплатно на arxiv.org.

4. Sutskever I. "Training Recurrent Neural Networks"

PhD Thesis, University of Toronto, 2013. Оптимизация для RNN Hessian-free optimization. Влиятельная работа от сооснователя OpenAI.

5. Zhang A., Lipton Z.C., Li M., Smola A.J. "Dive into Deep Learning"

Cambridge University Press, 2023. **Современный интерактивный учебник.** Практическая оптимизация для DL. Код на PyTorch/TensorFlow. Бесплатно: d2l.ai.

6. Kingma D.P., Ba J. "Adam: A Method for Stochastic Optimization"

ICLR, 2015 (arXiv:1412.6980). Оригинальная статья про Adam optimizer. Один из самых используемых алгоритмов. Обязательно для понимания современной практики.

Спасибо за внимание!

Каганов Юрий Тихонович

yurijkaganov@gmail.com

kaganov_y.t@bmstu.ru